

Exercise 3: Change What Claude Sees

Session: 3 — Context Engineering

Duration: 45 minutes, plus a 5-minute closing round

Project: The same BookStore API.

Goal

In session 2 you improved the words in your prompt. Now the prompt stays weak. You change the **context** instead: the files and the conversation that Claude sees. Then you check what that does to the code.

Two steps: write rules for this project in `CLAUDE.local.md`. Then give one prompt to a clean session and to a polluted session, and compare the code. The clean session also shows whether your rules work.

The tasks take 35 minutes. The rest of the time is for the bonus.

Before you start

Titles in *italics* are slide titles from this session.

Where to work. Open a terminal in the `bookstore` folder, the one that contains `pom.xml`. Start Claude with `claude`. Open a second terminal in the same folder now; task 2 uses it for the grader.

Check your starting point. After session 2, all tests pass and the work is committed. Run `./gradlew runTests` (or `mvn test`) and `git status` to check. If a test fails, or `git status` lists changes, run `/catch-up 2` in Claude. It finishes the session 2 work and commits. Before you run it, press Shift+Tab until the screen says auto mode, so it works without questions. Read on while it works. This exercise resets the project, and a reset removes every change that is not committed.

Fresh session. Type `/exit`, then start `claude` again. Do not use `/clear`: it does not always reload a changed `CLAUDE.local.md`. A fresh session remembers nothing and reads all context files again.

Training mode. The project's `CLAUDE.md` tells Claude to teach instead of answer. Before it explains or fixes something, it asks you one question. Tests, git commands and `/` commands are not affected. Stuck, or out of time? Say "*just tell me*". Claude then answers directly. That also works with the coach commands.

The experiment tag. An experiment must start right away, without a teacher question. So the experiment prompts in this sheet start with the experiment prefix from sessions 1 and 2. This sheet calls it the tag:

[Exercise 3 experiment – execute directly, no leading questions.]

Copy every prompt exactly as printed, tag included. Both sessions in a comparison get the same prompt.

Never edit `CLAUDE.md`. It carries training mode for sessions 4 to 8. Your own rules go in `CLAUDE.local.md`. Claude reads both files (*The CLAUDE.md Hierarchy*). `CLAUDE.local.md` is in `.gitignore`, so it stays on your machine.

Minute cue. The trainer calls "task 2" at minute 10. Move on then, also when your file is not finished. Commands that start with `/` and are not in Claude Code itself (`/catch-up`, `/context-coach`, `/save-changes`, `/verify-exercise`, `/pollute`) come with this project. Each task explains its command where it is used.

Tasks

1. Write `CLAUDE.local.md` (10 min)

You're going to build `CLAUDE.local.md`: the file Claude reads automatically at the start of every session in this project. `/init` writes you a first draft. You then go through it by hand, cut the lines that don't earn their place, and add the rules a new teammate would need but can't read off the code (*Start with `/init`, then refine by hand*).

1. **Run** `/init` (2 min). In this project it writes `CLAUDE.local.md`, not `CLAUDE.md`. Check with `git status`. If `CLAUDE.md` is listed: run `git diff CLAUDE.md`, copy the added lines into `CLAUDE.local.md`, then run `git checkout -- CLAUDE.md`.
2. **Delete every line that changes nothing** (4 min). For each line, ask: what would Claude do differently because this line exists? No answer: delete it (*Which CLAUDE.md Line Is Worth Its Tokens?*). Delete a line like "Write clean code". Keep the exact test command. Keep the line that says only the store touches the database. If there is none, add this one: "Every handler (`BookHandler`, `AuthorHandler`, `ReviewHandler`) calls the store. Only the store talks to the database."

3. **Add three team rules** (3 min). A rule is a decision your team made; Claude cannot read it from the code. Write each rule at the end of `CLAUDE.local.md`, in one or two sentences. (The `.claude/rules` folder comes later, in the "Back at work" section.) Rule 1 is written for you. Write rules 2 and 3 in the same style, in your own words:

- Rule 1: *"Handlers validate the request before they call the store. A handler never passes unchecked input to a store function."*
- Rule 2: every new endpoint comes with tests, in the same style as the existing tests. Open a test in `src/test/kotlin/bookstore/handler/` first to see the style: plain JUnit 5, `@Test` methods with `kotlin.test` assertions.
- Rule 3: no new external dependencies beyond `sqlite-jdbc`.

A rule must be checkable: someone who reads a diff can say "this breaks the rule". *"Keep dependencies minimal"* is not checkable. *"Never add a new library"* is.

4. **Run the test command from your file** (1 min). `/init` guessed it. Run it exactly as written. If the command does not run, fix the line.

5. **Optional: ask the coach, one round.** Only if you are done before minute 10. Run `/context-coach 1` (1 is the task number), then paste the whole file as your next message. The coach names the weakest line and asks one question. Fix that one thing, then go on.

Done when: the file has the lines you kept, a database line and three rules. Its test command runs. `git status` does not list `CLAUDE.md`.

2. Clean session versus polluted session (20 min)

In this exercise, you send the same prompt to two sessions and compare what they build. One session is fresh. The other you pollute first, with long answers, a pasted log, and a wrong fact. That's Session B from the slide *Which Session Is in More Trouble?*. Both sessions still read your `CLAUDE.local.md`, so the fresh session also tells you whether your rules hold up.

There are four sessions, run in this order. First, Clean, in the first terminal: it edits the project, so let it finish and save before you touch anything else. Then Grader, in the second terminal: start it and leave it running. It only reads. Then Polluted, back in the first terminal: it edits the project too, which is why Clean goes first. Last, Compare, in the second terminal, once Grader's report is done.

Session	Terminal	What you type, in this order	At the end
Clean	first	<code>/context</code> , the prompt, <code>/save-changes clean</code>	<code>/exit</code> , then start Grader in the second terminal
Grader	second	<code>/verify-exercise 3</code> , then your guess	leave it running, go back to the first terminal
Polluted	first	<code>/pollute</code> , the three messages it prints, <code>/context</code> , the prompt, <code>/save-changes polluted</code>	stays open
Compare	second	<code>/exit</code> , then <code>/context-coach 2</code> , then the two diff file names	you give six answers

1. **Write a prediction and a guess** (2 min). The prompt for both sessions is: add a DELETE endpoint for reviews, with tests. Write one sentence: which session builds the better endpoint, and what will differ. Example: *"I expect `ReviewHandlerV2.kt` or `review logic` in `BookHandler.kt` in the polluted diff."* A wrong prediction is fine. No prediction is the only failure. Then make one guess. Four lines from your file get tested this round: the database line, rule 1, rule 2, and rule 3. Which one will Claude break in the clean session? Write it down.

2. **Clean session** (4 min). In the first terminal, type `/exit` and start `claude` again. Run `/context` and note the percentage of the context window in use. Paste this exactly:

[Exercise 3 experiment – execute directly, no leading questions.] Add a DELETE `/reviews/{id}` endpoint to the BookStore API, with tests.

Watch which files Claude opens before it writes code. When it is done, run `/save-changes clean` . It saves all changes to the file `session3-clean.diff` and cleans the project; `CLAUDE.local.md` is not touched. Type `/exit` .

3. **Start the grader** (1 min). In the second terminal, start `claude` and run `/verify-exercise 3` . It grades `session3-clean.diff` against your `CLAUDE.local.md` . It only reads; it never edits the project. It first asks for your guess: type the name of the line, for example *rule 3* . Then leave it running and go back to the first terminal.

4. **Polluted session** (7 min). Start `claude` again in the first terminal.

1. Run `/pollute`. Claude writes three long cookie recipes and pastes the whole test output. This is Session B from the inside.
2. `/pollute` ends by printing three messages for you to send: a wrong fact about a file `ReviewHandlerV2.kt`, a correction, and a second wrong fact about `BookHandler.kt`. Copy and send them one at a time, and wait for each reply. If Claude wants to start editing, answer "*nothing to do yet*", without the tag.
3. Run `/context` and note the percentage.
4. Paste the same prompt as in step 2, with the tag. Wait until Claude is done.
5. Run `/save-changes polluted`. Keep this session open for the bonus.

5. **Compare** (6 min). Go to the second terminal. If the grader is still working, wait for it. Its report stays on the screen after you leave; task 3 scrolls up to it. Type `/exit`, start `claude` again, and run `/context-coach 2`. In your next message, type the two file names on one line:

`session3-clean.diff session3-polluted.diff`. The coach goes through three checks, one at a time:

- Did the change land in the right file?
- Does the new handler copy the existing handlers, including the status code the book DELETE returns?
- Is there nothing from the wrong fact or the pasted log?

For each check, the coach shows lines from both diffs next to the current

`src/main/kotlin/bookstore/handler/ReviewHandler.kt`. You answer *pass* or *fail* for both sessions, for example *clean: pass, polluted: fail*. Three checks, two sessions: six answers, each before the coach gives its own. Then tell the coach the two `/context` percentages and ask whether they explain the difference in the code.

Done when: `session3-clean.diff` and `session3-polluted.diff` exist. You gave six answers before the coach did. For every mistake, you wrote down the pollution step behind it; the coach names it.

3. Closing: read the grader's report (5 min)

Scroll up in the second terminal to the report from `/verify-exercise 3`. Write down three lines:

1. **Your guess.** The report says for each of your four lines whether Claude followed it (*held*), broke it (*failed*), or never needed it in this run (*not tested*). Write the result next to the line you guessed.

2. **The line that helped the most.** The report names one and shows the diff line it prevented or forced. Do you agree? One sentence.
3. **A line not worth its tokens.** The report names one line that changes nothing. Agree, or say what Claude does differently because of it. One sentence.

Done when: three lines are written down, each with one sentence of evidence. Bring them to the closing round.

Bonus (only if time remains)

Test your rules with a weak prompt. A DELETE endpoint rarely tempts Claude to add a library or to put the cache logic in the handler instead of the store. Caching does. Open a fresh session in the first terminal and paste this exactly. It is the vague prompt from session 2, unchanged:

```
[Exercise 3 experiment – execute directly, no leading questions.] Add caching to the BookStore API
```

When Claude is done, run `/save-changes rules`. Then, in the second terminal, run `/verify-exercise 3` again in a fresh session. It now grades `session3-rules.diff`, which tests all four lines. Compare its report with the first one: which lines were *not tested* before?

`/compact`, **then ask about the wrong fact.** In the polluted session, run `/compact`. Then ask, with the tag:

```
[Exercise 3 experiment – execute directly, no leading questions.] What do you know about ReviewHandlerV2.kt, and where does review logic live in this project?
```

If the wrong fact survived: `/compact` keeps what sounded important, including confident mistakes. If it is gone: `/compact` dropped something without asking you. Only a fresh session is a guaranteed reset.

Back at work: try this on your own project

Move one rule to its own file. The handler-validation rule only matters when Claude edits handler code (*Rule Discovery: With or Without Paths*). Move it to `.claude/rules/handlers.md` (create the folder) and remove it from `CLAUDE.local.md`. The file starts with these lines, then the rule:

```
---  
description: How handlers treat incoming requests  
paths: "src/main/kotlin/bookstore/handler/**/*.*kt"  
---
```

Check the pattern with `ls src/main/kotlin/bookstore/handler/*.kt`. A pattern with a small mistake matches nothing, and you get no error. Then ask for a small change in a handler. The rule appears in the session only then (*Progressive Disclosure*).

`/context-coach 4` reviews the file (4 is this item's number for the coach).

A README per folder. Put a `README.md` in `src/main/kotlin/bookstore/store/` with one instruction on its last line. Ask for a change in a store file, then in a handler. When does the instruction reach Claude?

Appendix: `/save-changes` by hand

If the command stops, it says why. Or run these from the project folder, one per line:

```
git add -A .  
git diff --cached --output=session3-polluted.diff  
git reset -q .  
git checkout -- .  
git clean -fd .
```

Replace `polluted` with `clean` or `rules`. The last command deletes files Claude created that you never committed. `CLAUDE.local.md` and the saved diffs are ignored by git and stay.

Closing round (5 min)

The trainer asks the room. Have these answers ready:

- How many `/init` lines did you keep?
- A line is true but changes nothing: which dimension does it hurt? A line is specific but wrong: which dimension does that one hurt?
- Which pollution step did the real damage? Did your prediction hold?
- One thing you would tell someone who skipped this session.